Prove Engineering Take Home: Supply Chain Negotiation

Build a small simulation where three AI agents negotiate deals across a supply chain, log structured traces of every decision they make, and build something that helps a human understand what happened and diagnose what went wrong in a run (if anything).

Time limit: 3 to 4 hours. AI coding tools (Claude Code, Codex, etc.) are expected and encouraged. Use whatever language or framework you want. Python or TypeScript preferred.

Whatever model you use for the LLM agents, please note it in your submission.

For the agent framework, you can use LangChain, LangGraph, n8n, or build it custom. Whatever you're fastest with. We don't care about the framework choice, we care about what you do with it.

The Simulation

Keep this simple. The marketplace is not the point. The traces are.

Setup: Three agents form a supply chain: a **supplier**, a **manufacturer**, and a **retailer**. The supplier sells raw materials to the manufacturer, who sells finished goods to the retailer. Each agent has a hidden reservation price (the minimum they'll accept or maximum they'll pay). Deals happen bilaterally (supplier↔manufacturer and manufacturer↔retailer) but outcomes cascade. The price the manufacturer pays the supplier determines what margin they have when negotiating with the retailer.

Each agent also has access to a `check_market_price(product)` tool that returns a reference price, but the tool returns slightly different values to each agent (simulating noisy or inconsistent market data). Agents don't know the others are seeing different numbers.

A run consists of one full supply chain negotiation: supplier↔manufacturer resolves first, then manufacturer↔retailer. The overall run succeeds if both deals close. It fails if either negotiation falls through.

You can vary the products, price ranges, and market conditions across runs. Up to you.

Agents: Each agent is an LLM with a small set of tools. They need ways to check market prices, make and respond to offers, close or walk away from deals, and communicate with other agents. How you design the tool set is up to you. The interesting design question is how much strategic tooling you give them. Do they get a way to review past offers? A way to reason about surplus? Those choices are part of the exercise.

Game loop: Within each bilateral negotiation, agents take turns. Each turn, an agent sees the current state of the negotiation and picks a tool call. Messages between agents are delivered on

the following turn, not instantly. A negotiation ends when one agent accepts, one rejects (walks away), or a round limit is hit.

Think about how tool calls can mimic failures in real world system behaviors. Your instrumentation should help surface when tool outputs don't match expectations, but not by just logging a tool error.

The agents don't need to be good negotiators. If they lowball, overpay, fall for bluffs, or walk away from deals they should have taken, that's fine. Bad negotiations often produce more interesting traces than clean ones. Don't spend time prompt engineering the agents to negotiate well. Spend it on the traces and the legibility layer.

In production multi agent systems, typically the hardest bugs aren't crashes. They're agents making reasonable decisions based on information that's no longer true. The cascading structure and inconsistent market signals create this problem by design.

The Traces

Integrate an observability tool (Langfuse, OpenTelemetry, or similar) to capture and export all agent traces. We want to see how the agents behaved end to end: tool calls, LLM inputs/outputs, latency, the full picture. Export the traces and include them in your submission.

On top of that, if you have time, also log a structured event record at each agent step. We are not prescribing a schema. Designing the right event model is part of the exercise. Consider what fields would actually help someone diagnose a failure after the fact across all runs, not just replay what happened in one trace manually. When a run fails, your traces should help someone determine whether the failure was in a single agent's decision, in the interaction between agents, or emergent from the system as a whole.

The Legibility Layer

Build something that helps a human answer three questions about a run:

1. **What happened?**
2. **Why did it happen?**
3. **What should change next?**

We're looking for something that supports diagnosis.

Could be a round by round replay showing how each agent's position shifted, a visualization of how prices converged (or didn't) across the chain, a timeline showing where agent beliefs about

market price diverged from each other and from reality, or something else entirely. What you build and why is a major part of what we're evaluating.

The cascading structure of the supply chain means a single negotiation transcript may not reveal the full picture. Think about how to show the relationships between deals.

If it has a visual component, keep it simple and lightweight. We're not looking for a design system. We are looking for intentionality: evidence that someone made deliberate choices about what to show and how, rather than just accepting whatever the AI generated. Default AI styling is easy to spot. We want to see that a human looked at the output, had opinions, and made it theirs. Think "someone built this with care on a deadline" not "an AI generated a dashboard."

Suggested Time Split

Time	Focus	Approximate Weight
~60 min	Simulation (supply chain, agents, negotiation loop)	20%
~75 min	Traces (structured logging, event schema)	35%
~60 min	Legibility layer (diagnosis tool)	30%
~30 min	Analysis, writeup, and cleanup	15%

These are guidelines. If you spend more time on traces and less on the simulation, that's fine.

What to Submit

1. **Your code**: GitHub repo (preferred) or zip. Commit at start and regularly. We review commit history to understand how you sequence your work and where you spend time. Don't squash into one commit at the end.
2. **Multiple runs** as structured JSON, hopefully with a mix of "successes" and "failures"
3. **A short Loom (1 to 3 min)** walking us through your decisions. Not a feature tour, just talk us through what you built and why.
4. **Your full AI conversation history**: export your chat logs from whatever tool you used. This is required and is some of the most important signal for us. We want to see how you think, how you direct the AI, and where you override it.

What We're Evaluating

In rough priority order:

- **Judgment**: did you make smart scoping decisions? Did you focus on the right things?
- **Trace quality**: is the structured data clean, complete, and well designed?
- **Legibility**: does your viewer/tool actually help a human understand what happened?
- **Taste**: does the output feel intentional or generated?
- **AI collaboration**: does your conversation history show someone driving the AI toward good outcomes, or just accepting defaults?

We are not evaluating raw coding ability (the AI can code), polish level, or how elaborate the marketplace is.